ShopOS Technical Documentation

Oct 29, 2026 · @Collins Jimu

1. Overview

ShopOS is a multi-tenant grocery and supermarket POS for Zimbabwe, built as a Spring Boot modular monolith on PostgreSQL with a React + TypeScript client that works offline. This document is the engineering reference for the MVP: how the system is structured, how data flows through checkout, sync and fiscalisation, and the rules every module must enforce.

Audience. The technical lead, the backend engineer (Java/Spring) and the frontend engineer (React/TypeScript). Product scope and commercial decisions live in the ShopOS PRD v1.0; this document covers only how the system is built.

Scope. P0 features for the first controlled pilot: tenancy and branches, catalogue, checkout, inventory, cashier shifts, fiscalisation, audit and offline recovery. P1 items (suppliers, batches, transfers, loyalty, consolidated dashboards) appear only where the design must leave room for them.

Non-negotiable engineering rules

1. The tenant is derived from the authenticated session, never from the request body.
2. Money uses `NUMERIC(19,4)` in PostgreSQL and `BigDecimal` in Java. No floating point.
3. Every write that a client may retry carries an idempotency key and is safe to replay.
4. Committed sales and fiscal records are never deleted. Corrections use returns, reversals and credit or debit notes.
5. A sale is not fiscalised until FDMS has accepted it. Sending a request is not success.
6. The product is not advertised as ZIMRA-compliant until testing, validation, registration and approval are complete.

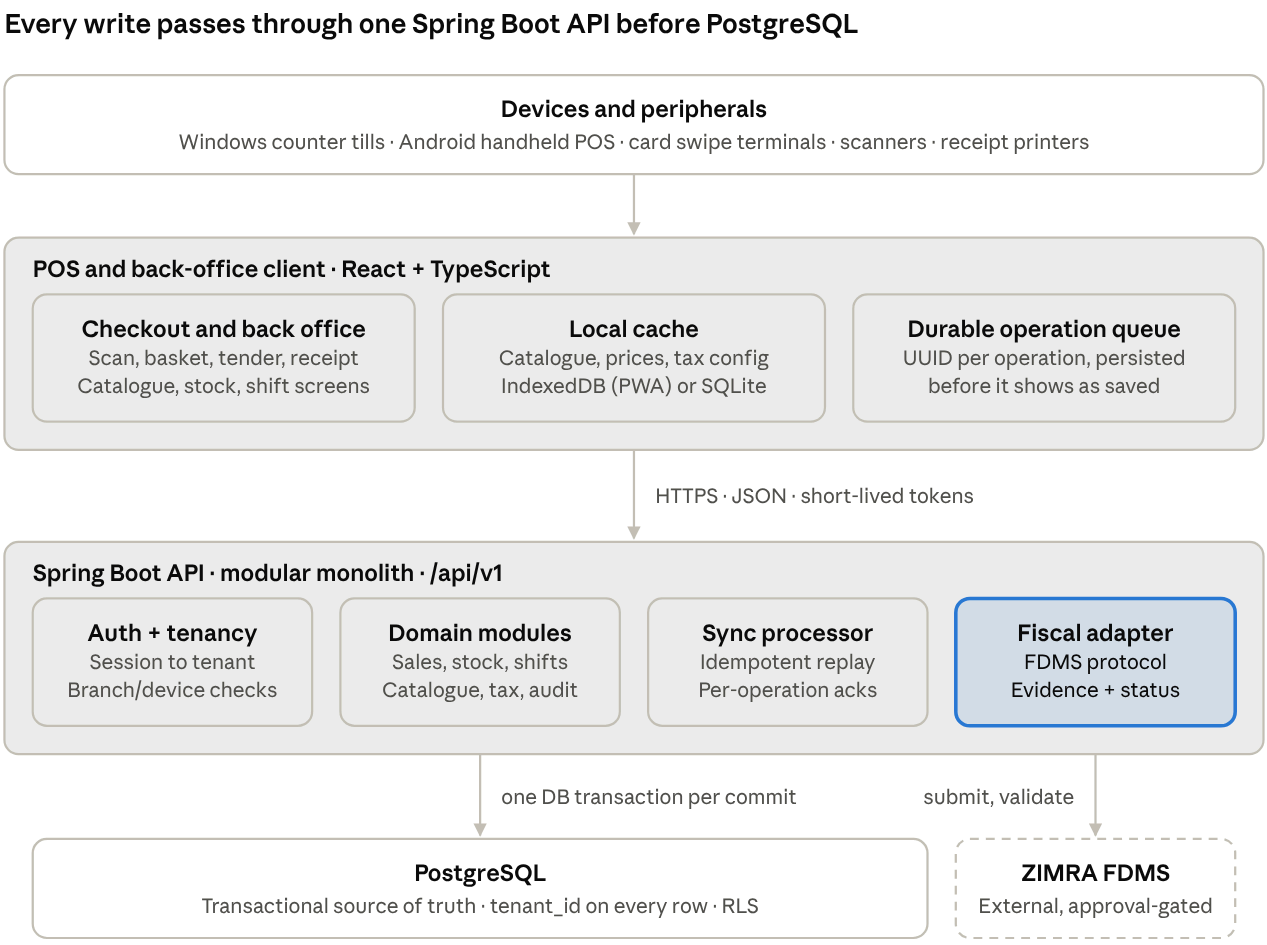
Conventions

Item	Convention
Identifiers	UUID v4, generated server-side except client operation IDs

Item	Convention
Timestamps	UTC (`TIMESTAMPTZ`); tenant display timezone defaults to `Africa/Harare`
Currency	ISO 4217 code on every monetary record; default `USD`
API	REST, JSON, base path `/api/v1`, OpenAPI documented
Migrations	Flyway, forward-only, reviewed by the technical lead
Status values	Upper snake case, enforced with `CHECK` constraints

2. System architecture

ShopOS is one deployable Spring Boot application in front of one PostgreSQL database, with an offline-capable client on every till. Microservices, message brokers and distributed deployments are deliberately excluded until the product has paying customers.



ShopOS system architecture · 4 layers, 1 external service

Tills talk only to the API; the API is the only writer to PostgreSQL and the only caller of ZIMRA FDMS, through the fiscal adapter.

Component responsibilities

Component	Responsibility	Must not
Checkout and back office	Capture scans, baskets, tenders and back-office edits	Compute authoritative totals, tax or fiscal IDs
Local cache	Hold the catalogue, prices, tax config and branch data the device is authorised for	Cache other branches or tenants

Component	Responsibility	Must not
Durable operation queue	Persist each operation with a UUID before showing it as saved; retry in order	Drop or reorder operations silently
Auth + tenancy	Resolve user, tenant, branch and device from the token; set DB tenant context	Trust a `tenantId` in the request body
Domain modules	Sales, payments, inventory, shifts, catalogue, tax, audit inside DB transactions	Call FDMS directly
Sync processor	Replay device batches idempotently and return a result per operation	Overwrite newer server prices, tax or permissions
Fiscal adapter	Map sale snapshots to FDMS requests, validate responses, store evidence	Invent a signature or mark a document fiscalised on send
PostgreSQL	Transactional source of truth with composite FKs and RLS	Hold money in floating-point types

Client packaging decision

Validate the real receipt printers and scanners before choosing packaging. Shared checkout logic and API contracts stay identical across options.

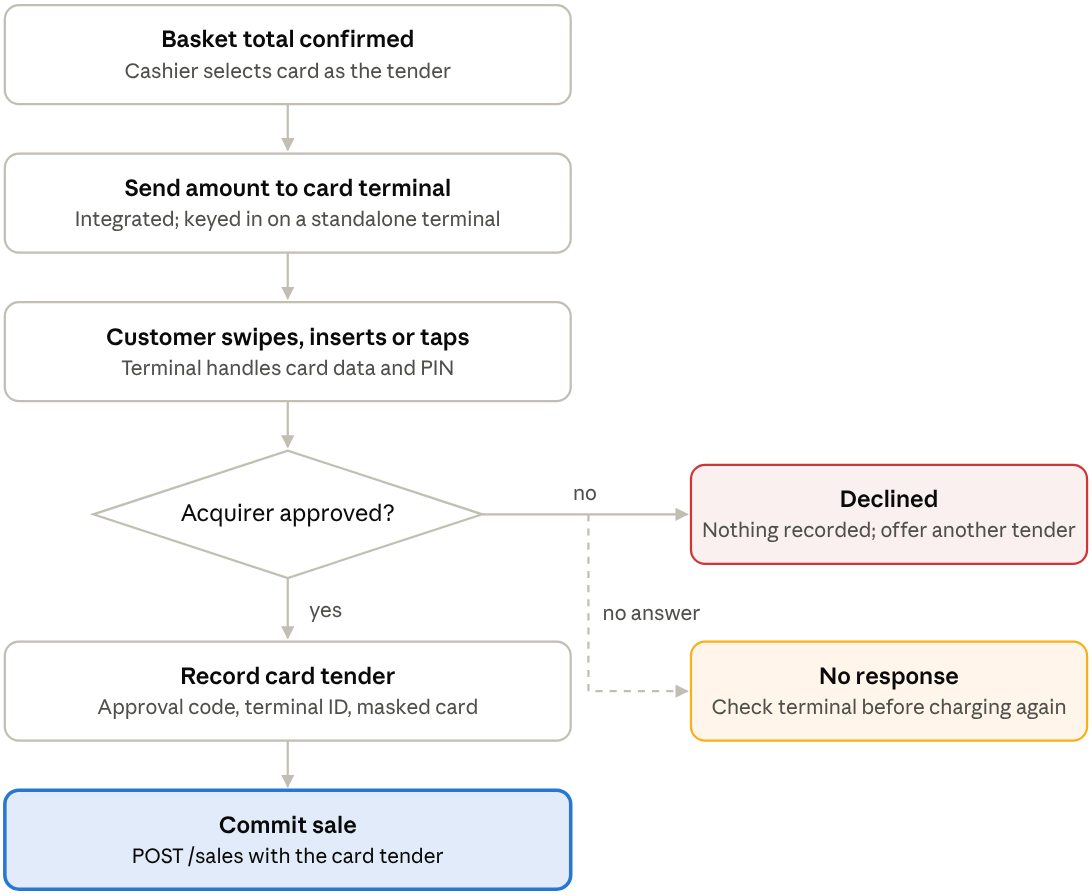
Platform	Default	Fallback if hardware fails
Windows counter till	Installable PWA	Tauri or Electron desktop wrapper for reliable printer control
Android mobile till	Installable PWA	Native wrapper
Back office	Responsive web	—

Handheld POS and card terminals

ShopOS supports Android handheld POS devices (with built-in printer, scanner and often a card reader) and card swipe terminals at the counter. Card data never passes through ShopOS: the terminal and the acquiring bank handle the card and PIN, and ShopOS stores only the result.

Device	Role	Integration
Android handheld POS	Mobile till for aisle selling, queue-busting, stock counts and deliveries	Runs the ShopOS client (PWA or native wrapper); built-in printer and scanner through the vendor SDK
Card swipe terminal, standalone	Takes card payments beside a counter till	Cashier keys the amount on the terminal and enters the approval code in ShopOS
Card terminal, integrated	Receives the amount from the till and returns the result	Vendor or acquirer SDK/API through a `PaymentTerminal` adapter in `payments`
Handheld with built-in card reader	Till and card terminal in one device	Same `PaymentTerminal` adapter, called on the device

A card tender is recorded only after the terminal approves it



Card payment flow · 1 decision, 2 failure paths

A terminal that does not answer is never treated as declined: the cashier checks the terminal or its last-transaction report before charging the card again.

Rules for card tenders

- Store only the approval code, terminal ID, provider reference, card scheme and masked card number (last four digits) in `sale_payments.provider_reference` and related fields; never the full card number, expiry or PIN.
- Add `CARD` to the payment methods and `ANDROID_HANDHELD` to the `pos_devices.platform` check, and register each card terminal against its branch.
- Card payments need the terminal's own connection to the acquirer; a till that is offline to ShopOS can still take a card if the terminal is online, and the sale queues as usual.
- Shift close reconciles card totals against the terminal's settlement (batch) report, separately from cash.

- Card refunds and voids go through the terminal first, then the ShopOS return or void with supervisor approval.
- Test every terminal and handheld model with the real acquirer before choosing the client packaging.

3. Backend module map

All modules live in one Spring Boot application under `zw.miccs.shopos`, each with its own package, public service interface and Flyway migrations. Modules call each other only through those service interfaces; no module reads another module's tables directly.

```
zw.miccs.shopos
├── identity          # users, credentials, sessions, tokens
├── tenancy           # tenants, memberships, tenant context, RLS setup
├── branches          # branches, tills, POS devices
├── catalogue         # products, barcodes, categories, CSV import
├── pricing           # selling prices, approved discounts
├── tax               # tax codes, effective-dated rate versions
├── inventory         # balances, movements, receipts, adjustments
├── sales             # sale commit, items, returns, voids
├── payments          # tenders, provider references
├── shifts            # cashier shifts, cash movements, reconciliation
├── fiscalisation     # fiscal devices, documents, attempts, fiscal days,
FDMS adapter
├── synchronisation   # bootstrap, batch replay, idempotency,
acknowledgements
├── reporting         # read models: sales, stock valuation, tax, cash
├── audit             # append-only audit events
└── subscriptions     # plans, entitlements, device and branch limits
```

Inside each module

Layer	Contents	Rule
`api`	REST controllers, request/response DTOs, OpenAPI annotations	No business logic; validate with Bean Validation
`application`	Service classes, `@Transactional` boundaries, permission checks	One public service interface per module

Layer	Contents	Rule
`domain`	Entities, value objects (`Money` , `Quantity`), invariants	No Spring or JPA imports where avoidable
`infrastructure`	JPA repositories, JDBC for row locking, external clients	Tenant-scoped queries only

Dependency rules

Module	May call	Never calls
`sales`	`catalogue` , `pricing` , `tax` , `inventory` , `payments` , `shifts` , `audit`	`fiscalisation` adapter internals
`fiscalisation`	`sales` (read snapshots), `tax` , `audit`	`inventory` , `payments` writes
`synchronisation`	`sales` , `inventory` , `shifts` service interfaces	Repositories of other modules
`reporting`	Read-only views of all modules	Any write
`audit`	Nothing (append-only sink)	—

Sales calculation and the FDMS protocol stay separate so the protocol can change without rewriting checkout. Enforce these rules with ArchUnit tests in CI.

4. Multi-tenant security

All tenants share one PostgreSQL database and schema, isolated by `tenant_id` on every tenant-owned row. Spring Security enforces access in the application; PostgreSQL Row-Level Security is a second safeguard, never the only one.

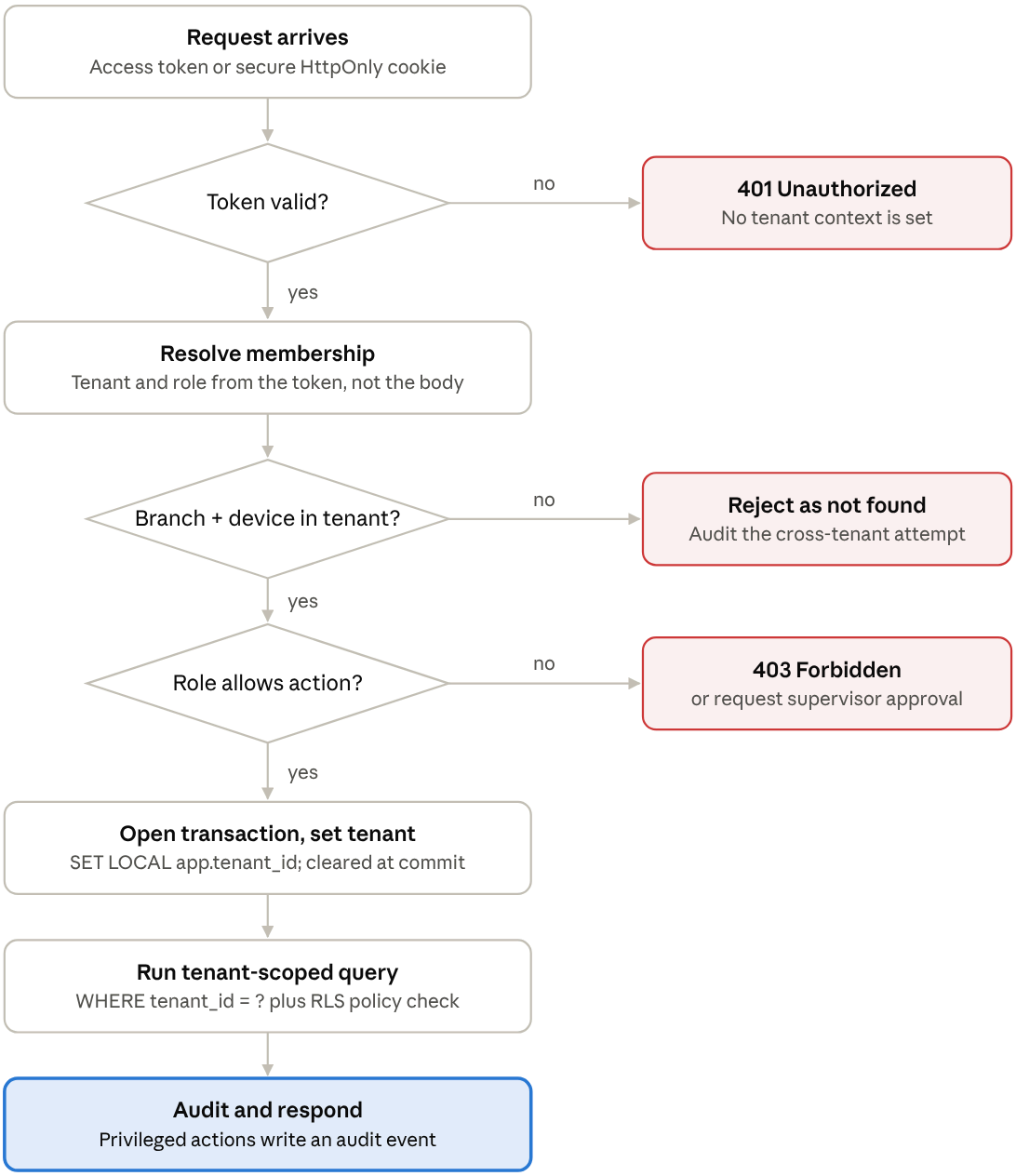
Tenant hierarchy

```
Platform
 └── Tenant (retail business)
      ├── Branches
      │    ├── Tills / POS devices
      │    ├── Inventory balances (branch-specific)
      │    ├── Cashier shifts
      │    └── Sales and fiscal records
      ├── Users and memberships (role per tenant)
      ├── Product catalogue (shared across branches)
      └── Subscription and plan
```

An independent shop is one tenant with one branch. A supermarket group is one tenant with several branches and tills.

Request authorisation flow

A request touches data only after its tenant checks pass



Request authorisation flow · 3 checks, 3 rejection paths

Every check runs on the server for every request, including sync batches; the client's filters are never trusted.

Setting tenant context safely

Set the tenant for the current transaction only, so a pooled connection never carries a stale tenant into the next request.

```
-- Run inside the transaction, before any tenant-owned query
SELECT set_config('app.tenant_id', :tenantId, true);  -- true = transaction-local

ALTER TABLE products ENABLE ROW LEVEL SECURITY;
ALTER TABLE products FORCE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON products
  USING (tenant_id = current_setting('app.tenant_id')::uuid)
  WITH CHECK (tenant_id = current_setting('app.tenant_id')::uuid);
```

Apply the same policy to every tenant-owned table, and run RLS tests with the real application database role, not a superuser.

Roles

Role	Typical rights
`OWNER`	Everything, including subscription and fiscal device registration
`ADMIN`	Users, branches, devices, catalogue, settings
`MANAGER`	Reports, stock adjustments, sync review screen
`SUPERVISOR`	Approve voids, returns, discounts and shift reconciliations
`CASHIER`	Open own shift, sell, record payments
`STOCK_CONTROLLER`	Goods received, wastage, counts
Platform support	Miccs staff; every access needs a stated reason and an audit record

Isolation tests (must pass in CI)

- [ ] Tenant A cannot read, update or list Tenant B records by direct ID.

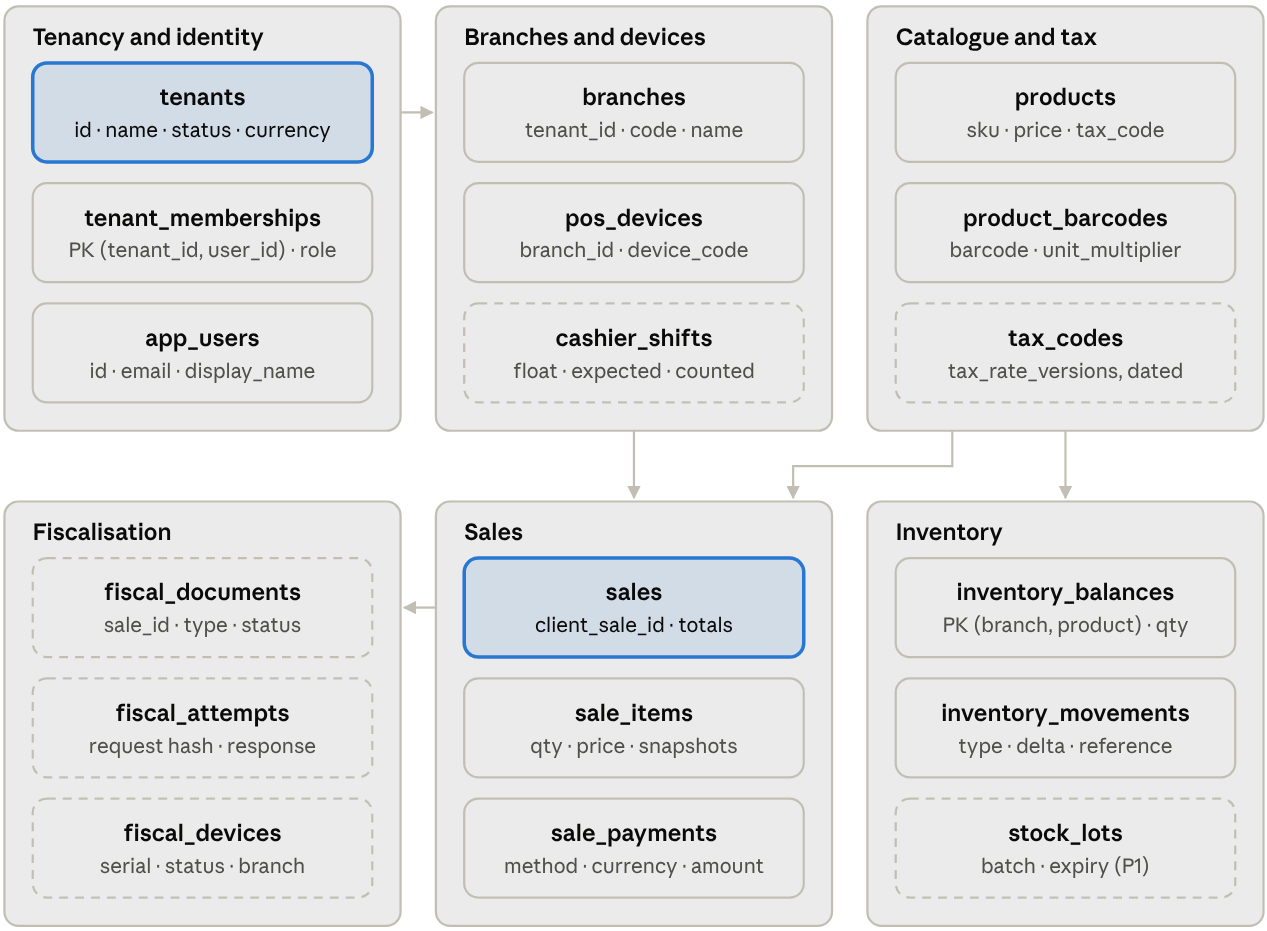
- [ ] A sale in Tenant A cannot reference a product, branch or device in Tenant B (composite FK rejects it).
- [ ] A sync batch replayed with Tenant B's device IDs under Tenant A's token is rejected.
- [ ] A `tenantId` in a request body is ignored.
- [ ] A pooled connection reused after a request has no tenant context.
- [ ] Fiscal credentials, tokens and secrets never appear in logs or API responses.

5. Data model

The schema is PostgreSQL with UUID keys, UTC `TIMESTAMPTZ` timestamps, `NUMERIC` money and composite `(tenant_id, id)` foreign keys so no row can reference another tenant's data. The PRD's core DDL is migration `V1`; the dashed tables below follow before production.

Every table hangs off a tenant; sales tie the rest together

Arrows run parent to child · dashed = planned before production · every table carries tenant_id



Core data model · 6 domains, 18 tables

A sale points to its branch, device and cashier; its items point to products; inventory and fiscal records point back to the sale that caused them.

Key constraints

Table	Constraint	Why
`sales`	`UNIQUE (tenant_id, device_id, client_sale_id)`	A replayed sale returns the existing row instead of a duplicate
`sales`	`UNIQUE (tenant_id, branch_id, receipt_number)`	Receipt numbers never repeat within a branch

Table	Constraint	Why
`sale_items`	`FK (tenant_id, product_id) → products(tenant_id, id)`	Blocks cross-tenant product references
`inventory_balances`	`PK (tenant_id, branch_id, product_id)`	One balance row per product per branch, lockable
`inventory_movements`	`CHECK (quantity_delta <> 0)`	Every movement changes stock; history is append-only
`product_barcodes`	`UNIQUE (tenant_id, barcode)`, `unit_multiplier > 0`	One barcode per tenant; pack barcodes sell multiples
`pos_devices`	`FK (tenant_id, branch_id) → branches`	A till belongs to exactly one branch of its tenant

Planned tables

Table	Purpose
`tax_codes`, `tax_rate_versions`	Effective-dated tax treatment; historic sales keep their snapshot
`cashier_shifts`, `cash_movements`	Opening float, paid in/out, expected and counted cash
`fiscal_devices`	Per-taxpayer device registration, serials, status, branch
`fiscal_documents`	Invoice, credit and debit note status and FDMS identifiers
`fiscal_attempts`	Attempt number, request hash, response code, redacted evidence
`fiscal_day_sessions`	Fiscal-day open/close state and reported results
`sync_operations`	Device operation IDs, idempotency keys, outcomes

Table	Purpose
`audit_events`	Actor, tenant, branch, action, reason, entity, timestamp
`subscriptions`, `plans`	Entitlements, billing state, device and branch limits
`suppliers`, `purchase_orders`, `goods_receipts`, `stock_lots`	P1 purchasing, receiving, expiry and batch tracking

Data rules

- Never delete completed sales or fiscal records; reverse them with returns, credit notes or corrections.
- Store immutable snapshots of SKU, description, unit price, tax code and currency on every sale line.
- Update stock with `SELECT … FOR UPDATE` on `inventory_balances` or an atomic `UPDATE … SET quantity = quantity - :q` inside the sale transaction.
- Never recalculate historic sales with a newly configured tax rate.
- Mixed-currency tenders need explicit conversion and reconciliation rules before they are enabled.
- Index `(tenant_id, branch_id, occurred_at)`, `(tenant_id, product_id)` and `(tenant_id, status)` once query plans justify them.

6. API reference

The API is REST over JSON at `/api/v1`, documented with OpenAPI, authenticated with short-lived access tokens and rotated refresh tokens (HttpOnly cookies for browser clients). These are ShopOS routes, not ZIMRA FDMS endpoint names.

Conventions

Concern	Rule
Tenant	Taken from the token; never accepted in the body or path

Concern	Rule
Idempotency	Retryable writes carry a client UUID (`clientSaleId`, sync `operationId`); a replay returns the original result
Money and quantity	Decimal strings in JSON (`"5.00"`, `"2"`) to avoid float rounding
Pagination	`?page=&size=&sort=`; responses carry `totalElements`
Timestamps	ISO 8601 UTC (`2026-10-29T09:30:00Z`)
Errors	RFC 7807 `application/problem+json` with a stable `code`

```
{
  "type": "https://docs.shopos.co.zw/errors/insufficient-stock",
  "title": "Insufficient stock",
  "status": 409,
  "code": "INSUFFICIENT_STOCK",
  "detail": "SKU 6001234 has 1 on hand; 2 requested",
  "traceId": "c1f9…"
}
```

Identity, tenancy and branches

Method	Endpoint	Purpose
`POST`	`/auth/login`	Authenticate
`POST`	`/auth/refresh`	Rotate session/token
`POST`	`/auth/logout`	Revoke session
`GET`	`/me`	Current user and memberships
`POST`	`/tenants`	Create business
`GET` / `PATCH`	`/tenants/current`	Read or update allowed tenant settings

Method	Endpoint	Purpose
`GET` / `POST`	`/branches`	List accessible branches / create branch
`GET` / `PATCH`	`/branches/{branchId}`	Read or update branch
`GET`	`/branches/{branchId}/devices`	List tills and devices

Products and inventory

Method	Endpoint	Purpose
`GET` / `POST`	`/products`	Search and paginate / create product
`GET` / `PATCH`	`/products/{id}`	Read or update product
`POST`	`/products/import`	Validated CSV import
`GET`	`/products/barcode/{barcode}`	Barcode lookup
`GET`	`/branches/{id}/inventory`	Stock balances
`POST`	`/inventory/receipts`	Receive goods
`POST`	`/inventory/adjustments`	Authorised stock adjustment
`GET`	`/inventory/movements`	Movement history
`GET`	`/inventory/low-stock`	Reorder candidates

Checkout, shifts and reports

Method	Endpoint	Purpose
`POST`	`/shifts/open`	Open cashier shift
`POST`	`/shifts/{id}/cash-movements`	Record cash paid in/out
`POST`	`/shifts/{id}/close`	Reconcile and close shift
`POST`	`/sales`	Commit a sale with an idempotency key
`GET`	`/sales` , `/sales/{id}`	Search / retrieve sales

Method	Endpoint	Purpose
`POST`	`/sales/{id}/returns`	Initiate authorised return
`POST`	`/sales/{id}/void-request`	Request an allowed void or reversal
`GET`	`/reports/sales`	Totals by date, branch and cashier
`GET`	`/reports/stock-valuation`	Stock valuation
`GET`	`/reports/cash-reconciliation`	Shift reconciliation
`GET`	`/reports/tax-summary`	Tax by configured classification

Fiscalisation and synchronisation

Method	Endpoint	Purpose
`GET` / `POST`	`/fiscal/devices`	List / register fiscal device configuration
`GET`	`/fiscal/documents/{id}`	Fiscal document status
`POST`	`/fiscal/documents/{id}/retry`	Retry an eligible failed or pending submission
`POST`	`/fiscal/credit-notes`, `/fiscal/debit-notes`	Create credit or debit note
`POST`	`/fiscal/days/open`, `/fiscal/days/close`	Fiscal-day operations where required
`GET`	`/fiscal/health`	Integration status and pending work
`GET`	`/sync/bootstrap`	Download authorised catalogue and settings
`POST`	`/sync/batch`	Submit device operations
`GET`	`/sync/operations/{id}`	Check one operation's result

Contract: `POST /sales`

The client sends what was scanned and tendered; the server derives prices, tax, tenant and totals from trusted data.

```
{
  "clientSaleId": "0f7d6f8d-8dbe-49e7-ae88-49b9f390ef10",
  "branchId": "branch-uuid",
  "deviceId": "device-uuid",
  "occurredAt": "2026-10-29T09:30:00Z",
  "configVersion": 42,
  "currency": "USD",
  "items": [{ "productId": "product-uuid", "quantity": "2" }],
  "payments": [{ "method": "CASH", "currency": "USD", "amount": "5.00" }]
}
```

Response `201 Created` (or `200 OK` with the same body on replay):

```
{
  "saleId": "sale-uuid",
  "receiptNumber": "HRE01-000123",
  "replayed": false,
  "totals": { "subtotal": "4.60", "discountTotal": "0.00", "taxTotal":
"0.60", "grandTotal": "4.60" },
  "change": "0.40",
  "inventory": { "status": "APPLIED" },
  "fiscal": { "status": "PENDING_SUBMISSION", "documentId": "fiscal-doc-uuid"
}
}
```

A `201` means the sale is committed, not that FDMS accepted it. The client must read `fiscal.status` before printing a fiscal receipt. Figures above are illustrative only.

7. Checkout and sale commit

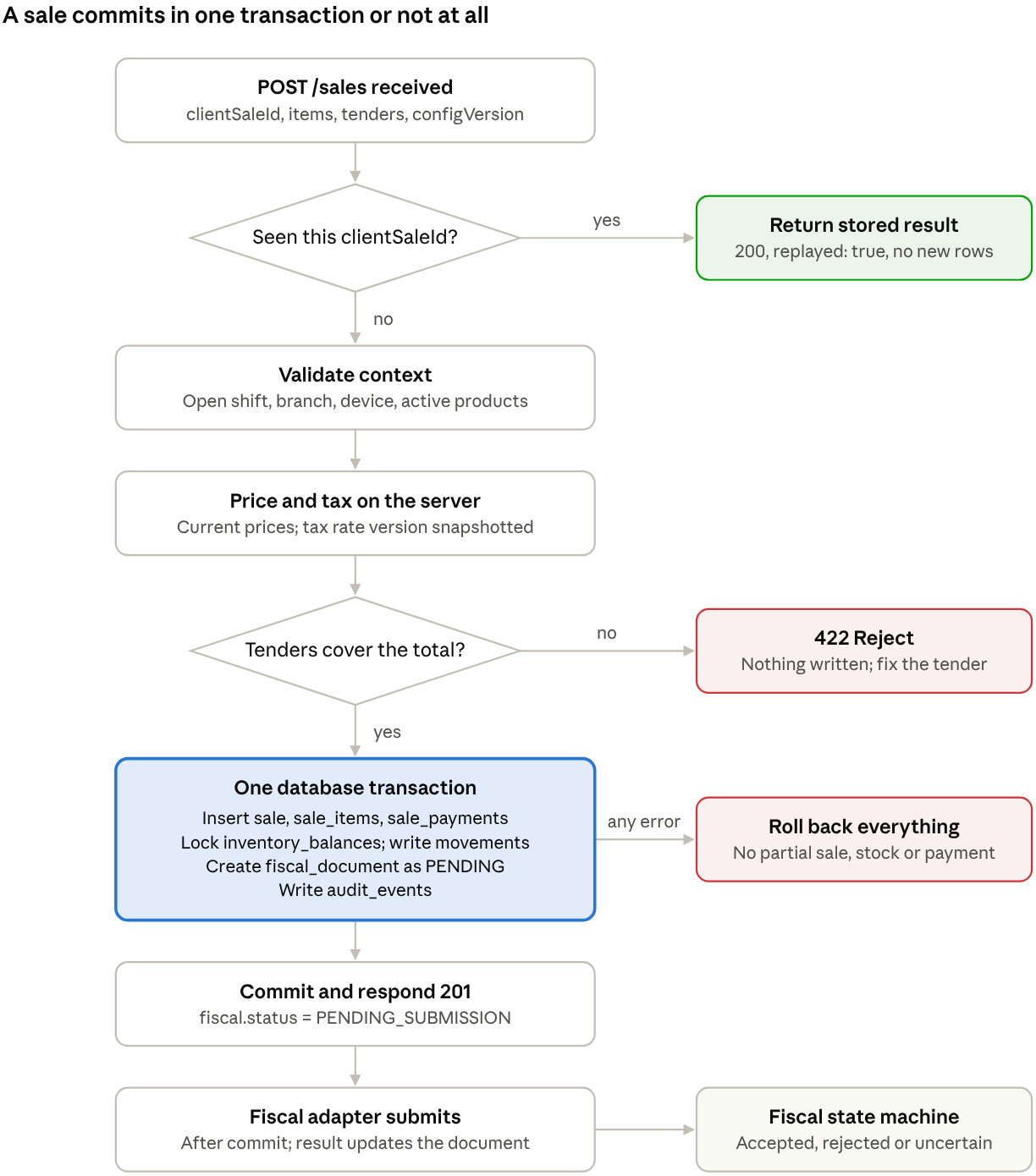
A sale is committed by one call to `POST /sales`, which either writes the sale, its lines, its payments, its stock movements and a pending fiscal document together, or writes nothing. Retries with the same `clientSaleId` return the original result.

Till-side checkout steps

1. Cashier opens a shift with an opening float (`POST /shifts/open`).

2. Cashier scans barcodes or searches; the client resolves each against the local cache and applies `unit_multiplier` for pack barcodes.
3. Cashier changes quantities and applies only approved discounts; supervisor approval is prompted where the role requires it.
4. Client shows indicative totals and collects tenders.
5. Client generates `clientSaleId`, writes the sale to the durable queue, then submits it.
6. Client prints a receipt only after reading the server's `fiscal.status` (see section 9).

Server-side commit



Server-side sale commit · 2 decisions, 1 transaction

The fiscal submission happens after the commit, so a slow or failed FDMS call never loses a sale or blocks the cashier from the next basket.

Implementation notes

- Catch the unique-constraint violation on `(tenant_id, device_id, client_sale_id)` and return the stored result; this covers two retries racing each other.
- Lock balance rows in a fixed order (by `product_id`) to avoid deadlocks between concurrent tills.
- Stock changes exactly once per committed sale: one movement row per line, referencing the sale ID.
- Receipt numbers come from a per-branch sequence inside the same transaction.
- Write the fiscal submission job in the same transaction (outbox pattern) so it cannot be lost if the process stops after commit.

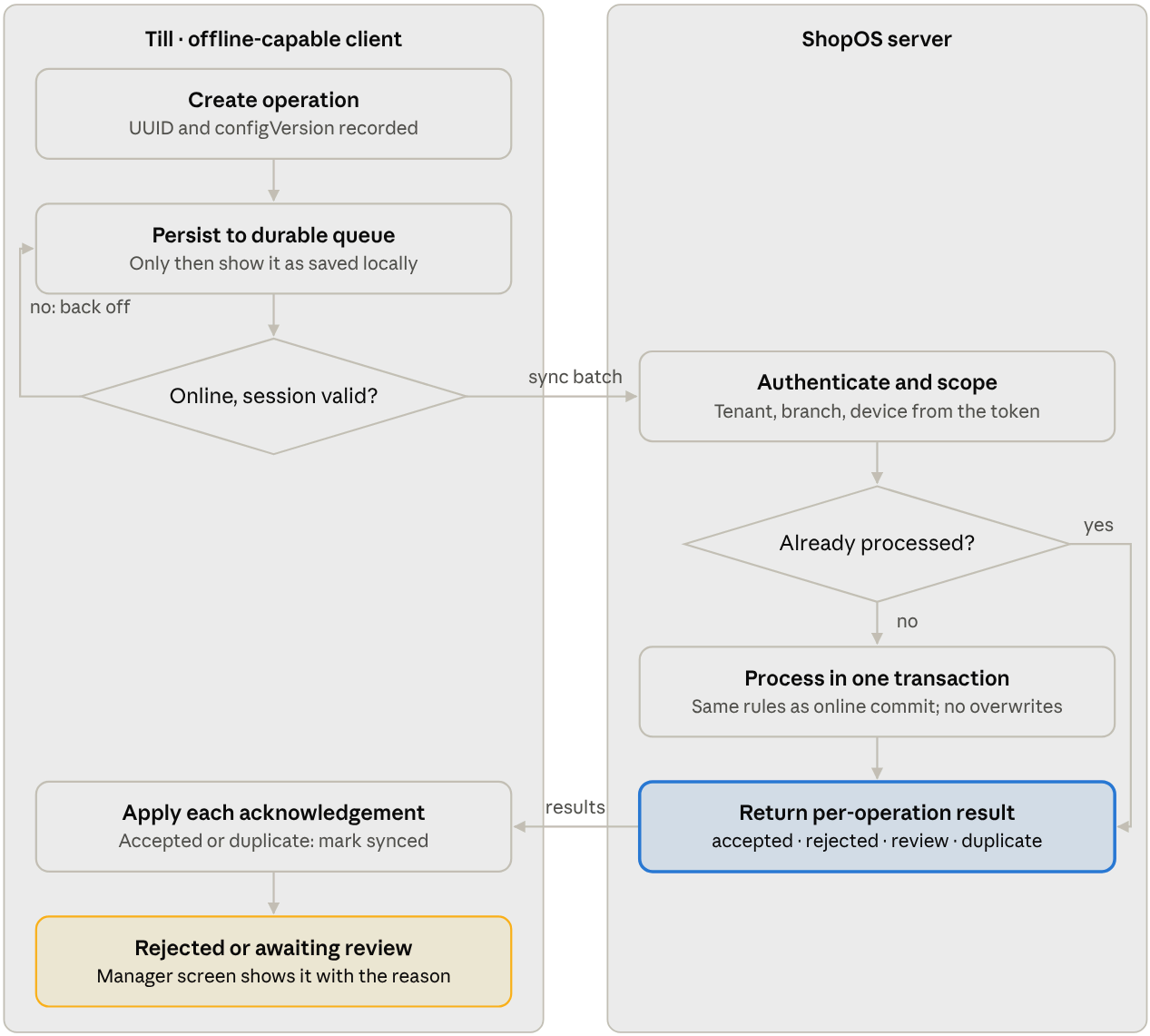
Definition of done for checkout

- [ ] A cashier opens a shift, scans products, sells a basket and records a payment.
- [ ] Totals and tax follow the approved tax configuration.
- [ ] Stock changes exactly once per committed sale.
- [ ] Receipt output follows the fiscalisation requirements for that transaction.
- [ ] Duplicate submissions create no duplicate sales or fiscal documents.
- [ ] A supervisor can inspect a void, return, discount or reconciliation adjustment.
- [ ] Another tenant cannot reach the sale through direct API calls, altered IDs or offline sync.
- [ ] The system recovers after a power interruption or network outage.
- [ ] Fiscal failures are visible, actionable and never treated as success.

8. Offline operation and synchronisation

A till keeps selling through a network outage by writing every operation to a durable local queue first, then replaying it to `POST /sync/batch` when it reconnects. The server answers every operation exactly once, and replays are harmless.

Every queued operation gets exactly one answer from the server



Offline queue and sync · 2 lanes, 4 outcomes

Operations are submitted in creation order; a rejected operation never blocks the rest of the batch, but it stays visible until a manager resolves it.

Local storage

Store	Contents	Refreshed by
Catalogue cache	Products, barcodes, prices for this branch	`GET /sync/bootstrap`, then deltas

Store	Contents	Refreshed by
Config cache	Tax configuration, permissions, branch settings, `configVersion`	Bootstrap and each successful sync
Operation queue	Pending sales, shift events, cash movements, each with UUID, sequence, retry count, status	The till itself
Device identity	`deviceId`, registered branch	Device registration

Use IndexedDB for a PWA and SQLite for a desktop or native wrapper. Encrypt local data where the platform supports it and require a device login.

Acknowledgement outcomes

Outcome	Meaning	Till action
`ACCEPTED`	Committed now	Mark synced, store server IDs
`ALREADY_PROCESSED`	Committed by an earlier attempt	Mark synced, store the original result
`REJECTED`	Fails a rule (inactive product, closed shift, wrong tenant)	Show on manager screen with the reason
`AWAITING_REVIEW`	Valid but needs a decision (price or stock conflict)	Show on manager screen; no automatic retry

Conflict rules

- Server prices, tax rules, permissions and product status always win; the till's view never overwrites them.
- Each operation records the `configVersion` it was created under, so a reviewer can see which price the cashier saw.
- Stock may go negative from offline sales; flag it for review rather than rejecting a sale the customer already paid for.
- Clock drift: the server stores both the device's `occurredAt` and its own receive time.

Fault-injection tests

- [ ] Upload interrupted mid-batch, then resumed.
- [ ] The same batch sent twice.
- [ ] Session expires while operations are queued.
- [ ] Device clock several hours wrong.
- [ ] Power cut between “persist” and “submit”, and between server commit and acknowledgement.
- [ ] Two tills selling the last unit of a product offline.

Offline fiscal receipts follow stricter rules, set out in section 9.

9. Fiscalisation

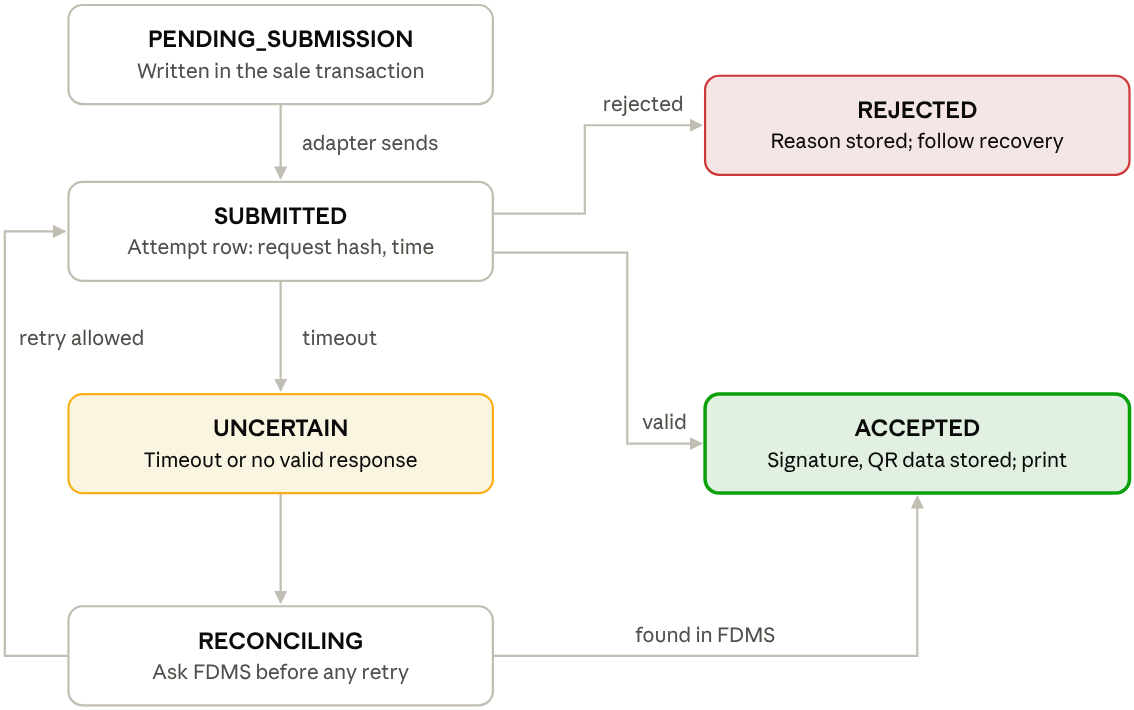
Fiscalisation is a launch gate: no live fiscal sales until ZIMRA testing, sample-document validation, approval and live registration are complete. The integration targets the ZIMRA FDMS virtual fiscal device API, with an approved hardware fiscal device as the alternative.

Adapter design

- `FiscalAdapter` is an interface in `fiscalisation`; `FdmsVirtualDeviceAdapter` is the first implementation, and a hardware-device adapter can follow without touching `sales`.
- The adapter reads the immutable sale snapshot and maps it to the exact FDMS request format in ZIMRA’s official API documentation and protocol version.
- It validates every response, then stores the signature, QR verification fields, FDMS identifiers and status on `fiscal_documents`.
- Every attempt is a `fiscal_attempts` row: attempt number, request hash, response code, timestamps and redacted evidence. Attempts are never deleted.
- Device credentials and certificates are encrypted at rest and never logged.

Document state machine

A fiscal document is accepted only on a valid FDMS response



Fiscal document state machine · 6 states

A timeout is never treated as a rejection or a success: the adapter reconciles with FDMS first and retries only when the protocol and the document's state allow it.

Credit notes and debit notes are separate fiscal documents that move through the same states and reference the original invoice.

Offline fiscal rule

Until ZIMRA or the approved integration partner confirms an offline workflow, the till must not issue a purported fiscal invoice when fiscalisation cannot complete. It may hold the sale as pending or use an approved fallback, and any non-fiscal slip must say so plainly.

Work packages

Package	Output	Exit criterion
A. Regulatory and protocol discovery	Current API spec, test credentials, registration process, test cases, device model for supermarket tills	Written answers to the open questions in section 12
B. Adapter implementation	Request mapping, response validation, persisted signatures and QR fields	Protocol-level automated tests pass against the test environment
C. Failure handling and reconciliation	Timeout, rejection, duplicate, credential-expiry and outage handling	Fault-injection tests pass; no ambiguous retry
D. Validation and approval	Test invoices, credit and debit notes, fiscal-day operations, sample submission	ZIMRA approval and live registration obtained

Fiscal-day open and close run through `/fiscal/days/open` and `/fiscal/days/close` where the protocol requires them, and `/fiscal/health` shows pending and failed documents to managers. Sources: ZIMRA Fiscalisation Explained and the Fiscalisation API documentation, as cited in the PRD.

10. Cashier shifts and inventory movements

Every cash sale belongs to an open shift, and every stock change is an append-only movement row; balances are a running total of movements, never edited directly.

Shift lifecycle

1. **Open.** Cashier declares the opening float on a device; status `OPEN`. One open shift per cashier per device.
2. **Trade.** Cash sales, refunds and cash movements (paid in, paid out, drops) attach to the shift.
3. **Count.** Cashier enters the counted cash per currency; status `COUNTED`. The till hides the expected figure until the count is entered.

4. **Reconcile.** Server computes the variance; a supervisor reviews and approves it with a reason; status `CLOSED`.
5. **Locked.** A closed shift accepts no new transactions; corrections are new, audited adjustments.

```
\text{expected cash} = \text{opening float} + \text{cash sales} - \text{cash refunds} + \text{paid in} - \text{paid out} - \text{drops}
```

```
\text{variance} = \text{counted cash} - \text{expected cash}
```

Compute both per currency when multi-currency tenders are enabled; never net USD against another currency without a defined conversion rule.

Inventory movement types

`movement_type`	Sign	Created by	Approval
`OPENING`	+	Initial stock load	Manager
`RECEIPT`	+	`POST /inventory/receipts`	Stock controller
`SALE`	−	Sale commit, one per line	Automatic
`RETURN`	+	`POST /sales/{id}/returns`	Supervisor
`WASTAGE`	−	Damaged or expired goods	Stock controller
`ADJUSTMENT`	±	`POST /inventory/adjustments` after a count	Manager, with reason
`TRANSFER_OUT` / `TRANSFER_IN`	− / +	Branch transfer (P1)	Manager

Each movement stores `reference_type` and `reference_id` (the sale, receipt or adjustment that caused it), so `inventory_balances.quantity` can always be rebuilt from history. A nightly job compares the two and alerts on any difference.

11. Testing, deployment and operations

Nothing merges to `main` without passing unit, integration, tenant-isolation and architecture tests in CI; nothing reaches production without running in staging first.

Test layers

Layer	Tool	Covers
Unit	JUnit 5	Money and tax calculation, state machines, domain invariants
Integration	Testcontainers (real PostgreSQL)	Transactions, locking, RLS policies, Flyway migrations, idempotent replays
Architecture	ArchUnit	Module dependency rules from section 3
Contract	OpenAPI schema checks	Request and response shapes the client relies on
Frontend	React Testing Library	Checkout screen, queue states, fiscal status messages
End to end	Playwright	Open shift, sell, go offline, reconnect, close shift
Fault injection	Scripted network and power cuts	Sync and fiscal recovery tests in sections 8 and 9
Hardware	Manual test sheet	Each supported printer and scanner on Windows and Android

Environments and pipeline

Environment	Purpose	Data	Fiscal endpoint
Local	Development	Seeded dummy tenants	Mock adapter
CI	Automated tests per pull request	Ephemeral Testcontainers	Mock adapter
Staging	Pilot rehearsal, demos	Anonymised or dummy	FDMS test environment
Production	Live pilots	Customer data	FDMS live, only after approval

Build one Docker image per commit and promote the same image from staging to production. Flyway migrations run on start-up and must be backward compatible with the previous release.

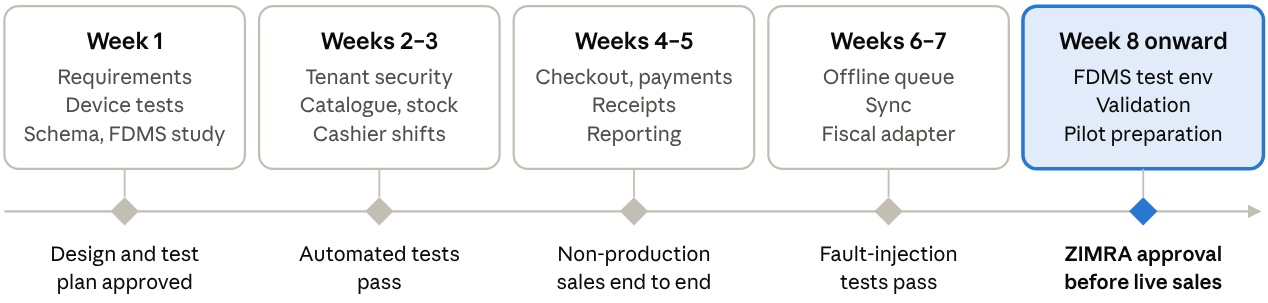
Operations

- **Monitoring:** structured JSON logs with `traceId`, `tenantId` and `deviceId`; error tracking; `/actuator/health` checks; alerts on failed fiscal documents, stuck sync queues and database errors.
- **Backups:** encrypted daily PostgreSQL backups plus point-in-time recovery; a restore is tested before the first pilot and then on a schedule.
- **Secrets:** fiscal credentials, signing keys and tokens in a secrets store, never in the repository or logs.
- **Access:** production database access restricted to named staff; platform support access to customer data requires a reason and an audit record.

12. Delivery plan, ownership and open questions

The build runs about eight weeks to a fiscal test environment, but live fiscal sales start only after ZIMRA approval; if registration, validation or hardware takes longer, the timeline extends rather than the gate moving.

Live fiscal sales wait for ZIMRA approval, whatever the schedule



Delivery roadmap · 5 phases, 5 exit gates (not to scale)

Each phase ends at its gate; the next phase starts only when the gate's criterion is met.

Ownership

Area	Owner	Deliverables
Backend, API, database	Intern 1 (Java)	Spring Boot structure, migrations, tenancy and authorisation, catalogue, tax, sales, payments, shifts, inventory, sync endpoints, fiscal adapter, OpenAPI, integration tests
Client, checkout, back office	Intern 2 (React)	Login and branch selection, checkout with barcode and keyboard, catalogue and imports, shift and reconciliation screens, offline cache and queue UI, fiscal status messages, Android layouts, Windows printer and scanner tests
Architecture, compliance, product	Technical lead	Approve schema, security and API contracts; ZIMRA liaison and fiscal device decisions; PR review; deployment and backups; pilot customers

Both engineers work to the same OpenAPI contract, agreed before each phase starts.

Open questions for ZIMRA or the integration partner

- [ ] May a fiscal document be issued while disconnected, and under what conditions?
- [ ] How must signatures, counters, sequence numbers and timestamps be handled?
- [ ] What happens when a submission is delayed or rejected?
- [ ] Is a registered hardware fiscal device required as a fallback?
- [ ] How does fiscal-day closure work after a network outage?
- [ ] Which fiscal device model fits a supermarket with several tills?

Open engineering decisions

- [ ] PWA or desktop wrapper on Windows, after printer and scanner tests.
- [ ] Whether mixed-currency tenders are allowed in the pilot, and the conversion rule.
- [ ] Policy for selling below zero stock while offline.
- [ ] Hosting region and provider for production and backups.

- [ ] Which handheld POS models and card terminals pilots will use, and which acquiring banks support integrated mode.
- [ ] Whether card payments start in standalone mode (approval code keyed in) or integrated mode.

Go-live checklist

- [ ] Interview at least 10 prospective retail customers.
- [ ] Approve database schema and tenant isolation design.
- [ ] Confirm FDMS requirements and test-environment access.
- [ ] Test actual Windows and Android printers and scanners.
- [ ] Implement checkout, inventory and shift reconciliation.
- [ ] Complete offline replay and recovery tests.
- [ ] Complete fiscal test documents and required approval.
- [ ] Test backups and restore procedures.
- [ ] Sign pilot agreements and define support arrangements.
- [ ] Obtain authorisation before enabling live fiscal sales.